

GLAM SingleAssetVault Audit Report

Table of Contents

1. Executive Summary

About GLAM

Audit Summary

Overall Security Posture

Launch Recommendations

Audit Scope

2. Assumptions and Considerations

3. Severity Definitions

Impact

Likelihood

Severity Classification Matrix

4. Findings

L01: Close mint ignores SingleAssetVault queue

L02: SingleAssetVault skips pending-queue gate in update_mint_apply_timelock.rs

L03: Jupiter Swap is not blocked for SingleAssetVault

5. Enhancement Opportunities

E01: SingleAssetVault is a subtype of TokenizedVault

E02: Invalid ATA owner priced as zero

E03: SingleAssetVault decimals are not constrained on init

E04: system_transfer is not blocked for SingleAssetVault

About Us

About Adevar Labs

Audit Methodology

Confidentiality Notice

Legal Disclaimer

3

3

3

3

4

4

5

6

6

6

6

8

8

10

12

13

13

15

16

17

18

18

18

19

20

1. Executive Summary

About GLAM

GLAM is a protocol for programmable investment infrastructure on Solana. It provides composable primitives like vaults, mints, integrations, policies, and permissions to launch tokenized products and onchain strategies with precision and control.

Audit Summary

This audit reviews a single commit (diffed against its immediate predecessor). That commit adds the `SingleAssetVault` type, a tokenized vault that is priced solely by its base asset.

The table below summarizes identified vulnerabilities by risk level and remediation status.

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	3	3	0

Enhancement Opportunities: 4

Overall Security Posture

This section describes the security posture before and after the fix review. The fix review is the stage of the audit where Adevar Labs verified the fixes implemented by the GLAM development team for the issues identified during the audit.

After Fix Review Summary

All findings identified in this audit scope were fixed, and all enhancement recommendations were implemented. The remediation improves maintainability by introducing shared helper functions for account-type checks. This makes the intent clearer, reduces duplicated conditional logic, and lowers the probability of regressions in future updates involving new account types. The team also addressed a prior assumption around integration behavior. Instead of allowing integrations to appear configurable but effectively unusable, the code now explicitly prevents activation where not allowed (in `SingleAssetVault` integration paths), which removes ambiguity and strengthens policy enforcement.

Before Fix Review Summary

Most issues are guardrails around admin-only flows (queue handling, mint initialization, and pricing). Two findings are best treated as enhancements that reduce the chance of future regressions by unifying `SingleAssetVault` and `TokenizedVault` behavior. The enhancements, if adopted, would also address several low-severity issues by default.

Launch Recommendations

- Treat **SingleAssetVault** the same as **TokenizedVault** in shared code paths (queue handling, timelock apply, pricing invariants), ideally via a single helper.
- Define the intended **SingleAssetVault** workflows and align policy enforcement across pricing, swaps, and native transfers.

Audit Scope

- **Repository:** <https://github.com/glamsystems/glam>
- **Before Fix Review Commit Hash:** **f888eee5273476146438e9782e8356cf3db22435**
- **After Fix Review Commit Hash:** **034ec1fc7709ddc7e3620f6a2d5d45d5909c2654**

2. Assumptions and Considerations

- **SingleAssetVault** is intended to be base-asset-only (no swaps into other assets and no native SOL transfers that move value outside the base asset).

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product’s reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

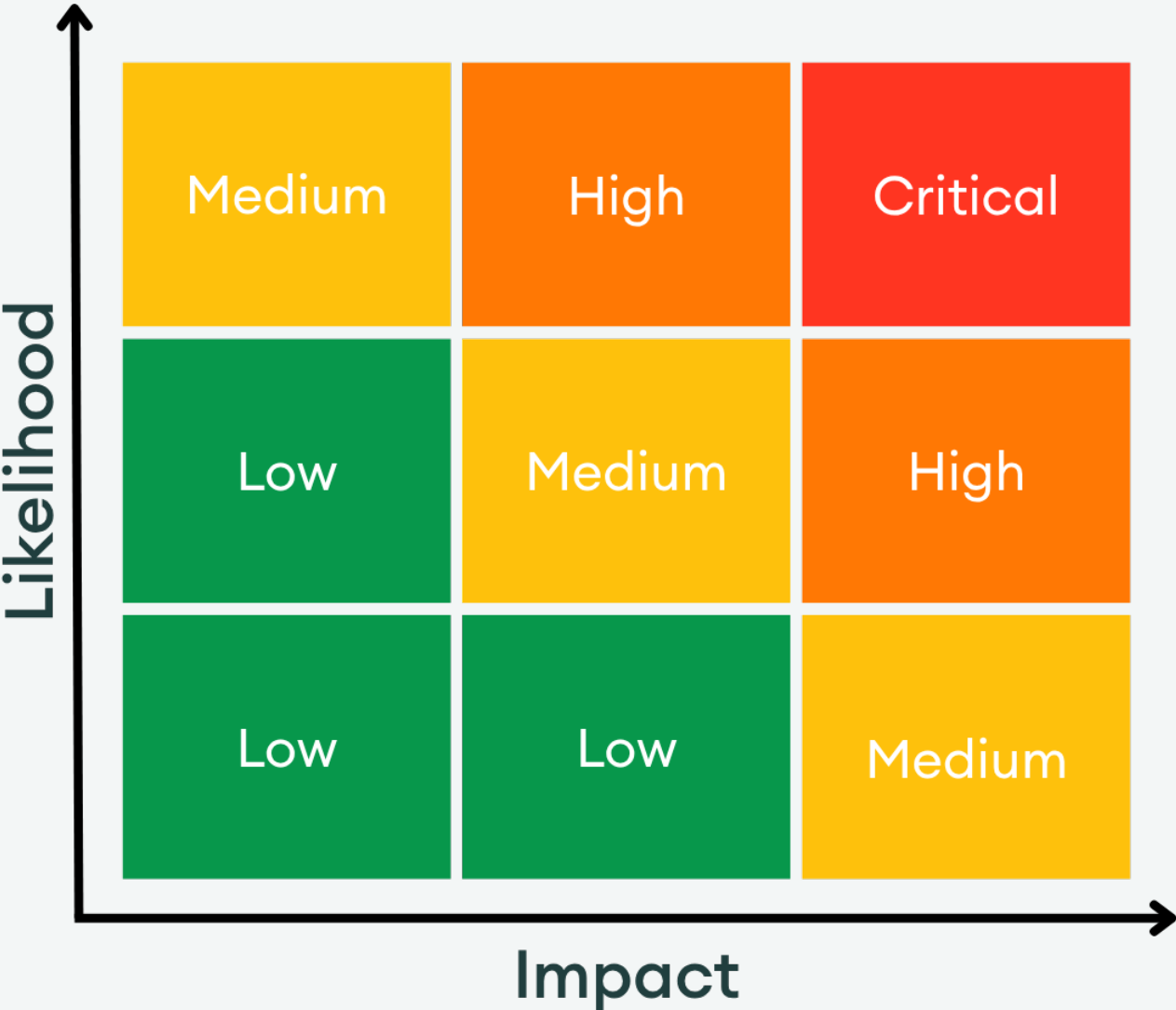
- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

L01: Close mint ignores **SingleAssetVault** queue

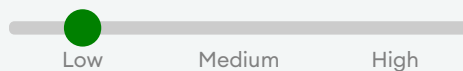
Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/glamsystems/glam/blob/f888eee5273476146438e9782e8356cf3db22435/anchor/programs/glam_mint/src/instructions/close_mint.rs#L22-L38

```
>_close_mint.rs RUST
20:
21:     let glam_state = &mut ctx.accounts.glam_state;
22:     if glam_state.account_type == AccountType::TokenizedVault {
23:         require!(
24:             ctx.accounts.request_queue.is_some(),
25:             GlamError::MissingAccount
26:         );
27:         let request_queue_ref = RequestQueueRef::from_account_data(
28:             ctx.accounts
29:                 .request_queue
30:                 .as_ref()
31:                 .unwrap()
32:                 .try_borrow_mut_data()?,
33:         )?;
34:         require!(
35:             request_queue_ref.count == 0,
36:             MintError::RequestQueueNotEmpty
37:         );
38:     }
39:
40:     // Close extra metas account
```

Description:

When closing a **SingleAssetVault** mint, the request queue is not checked, so pending requests could be stranded while the mint is closed.

Recommendation:

Check the requests count for **SingleAssetVault** to make sure there are none when closing the mint.

Developer Response:

Valid, mint of a single asset vault shouldn't be closed when the request queue is not empty.

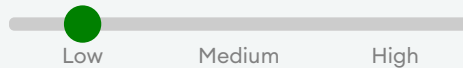
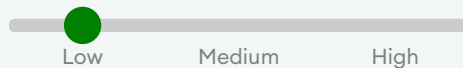
Mitigation Analysis:

Fixed according to the recommendation.

L02: SingleAssetVault skips pending-queue gate in update_mint_apply_timelock.rs

Status:

Resolved

Impact:**Likelihood:****Severity:**

Low

Location: https://github.com/glamsystems/glam/blob/f888eee5273476146438e9782e8356cf3db22435/anchor/programs/glam_mint/src/instructions/update_mint_apply_timelock.rs#L26-L46

```
> _update_mint_apply_timelock.rs RUST
24:     );
25:
26:     if glam_state.account_type == AccountType::TokenizedVault {
27:         // Disallow applying pending changes if there's any unfulfilled subscription o
r redemption
28:         // Manager should fulfill all unfulfilled subscriptions and redemptions before
applying changes
29:         require!(
30:             ctx.accounts.request_queue.is_some(),
31:             GlamError::MissingAccount
32:         );
33:         let request_queue_ref = RequestQueueRef::from_account_data(
34:             ctx.accounts
35:                 .request_queue
36:                 .as_ref()
37:                 .unwrap()
38:                 .try_borrow_mut_data()?,
39:         )?;
40:         for i in 0..request_queue_ref.count {
41:             let request = request_queue_ref.get(i)?;
42:             if request.is_pending_fulfillment() {
43:                 return err!(GlamError::CannotApplyChanges);
44:             }
45:         }
46:     }
47:
48:     // Get staged lockup period in glam_state.state_update_next.integration_acls
```

Description:

Pending updates can be applied even if a **SingleAssetVault** has pending queue items, unlike **TokenizedVault**.

Recommendation:

Check for pending requests before applying updates for **SingleAssetVault** type too.

Developer Response:

Valid, single asset vaults should enforce the same check that when timelock is applied there shouldn't be any unfulfilled requests in the queue.

Mitigation Analysis:

Fixed according to the recommendation.

L03: Jupiter Swap is not blocked for SingleAssetVault

Status:

Resolved

Impact:

Low

Medium

High

Likelihood:

Low

Medium

High

Severity:

Low

Location: https://github.com/glamsystems/glam/blob/f888eee5273476146438e9782e8356cf3db22435/anchor/programs/glam_protocol/src/instructions/jupiter_swap.rs#L281-L290

>_jupiter_swap.rs

RUST

```
279: }
280:
281: #[glam_macros::glam_vault_signer_seeds]
282: pub fn swap_handler<'c: 'info, 'info>(
283:     ctx: Context<'_, '_, 'c, 'info, JupiterSwap<'info>>,
284:     data: Vec<u8>,
285: ) -> Result<()> {
286:     check_integration(
287:         &ctx.accounts.glam_state,
288:         &crate::ID,
289:         SupportedProtocols::JupiterSwap.bits(),
290:     )?;
291:
292:     let params = JupiterIxParams::from_ix_data(&data, ctx.remaining_accounts)?;
```

Description:

`jupiter_swap` directly invokes Jupiter and does not guard against `SingleAssetVault`. If the owner enables JupiterSwap in `integration_acls`, they can swap the base asset into other tokens. Because `price_single_asset_vault` only prices the base-asset ATA, this can underprice AUM.

Recommendation:

Add a check `check_not_state_type` for `SingleAssetVault` similar to how it is done in `cpi_proxy`.

Developer Response:

Valid, there isn't a use case of Jupiter Swap for single asset vaults yet.

Mitigation Analysis:

Fixed according to the recommendation.

5. Enhancement Opportunities

E01: `SingleAssetVault` is a subtype of `TokenizedVault`

Location: https://github.com/glamsystems/glam/blob/f888eee5273476146438e9782e8356cf3db22435/anchor/programs/glam_protocol/src/state/accounts.rs#L180-L187

```
> _accounts.rs RUST
178: }
179:
180: #[derive(AnchorDeserialize, AnchorSerialize, Clone, Debug, PartialEq, Copy)]
181: pub enum AccountType {
182:     Vault,
183:     TokenizedVault,
184:     Mint,
185:     SingleAssetVault,
186:     // ... more account types
187: }
188:
189: #[derive(AnchorDeserialize, AnchorSerialize, Clone, Debug)]
```

Description:

Several findings stem from `SingleAssetVault` being modeled as a separate `AccountType` rather than a subtype of `TokenizedVault`. This makes it easy to miss shared invariants (queue checks, decimals, etc.) when branching only on `TokenizedVault`. This is an enhancement/clarity issue.

Another function, `validate_args`, currently checks `account_type != AccountType::Vault`. This negative check is easy to miss when new account types are added (eg, `SingleAssetVault`), and the logic is duplicated in other places. A helper like `supports_mint_init()` (or `is_tokenized_vault()` / `uses_request_queue()`) makes intent explicit and reduces the risk of future omissions.

Potential Benefit:

Improves clarity and reduces duplicated conditionals, lowering the risk of missing shared checks in future changes.

Recommendation:

```
RUST
#[derive(AnchorDeserialize, AnchorSerialize, Clone, Debug, PartialEq, Copy)]
pub enum TokenizedVaultKind {
    MultiAsset,
    SingleAsset,
}

#[derive(AnchorDeserialize, AnchorSerialize, Clone, Debug, PartialEq, Copy)]
pub enum AccountType {
    Vault,
    TokenizedVault(TokenizedVaultKind),
    Mint,
}
```


Something like the enum above would be ideal, but it is not backwards compatible. A lighter-weight alternative is:

```
impl AccountType {  
    pub fn is_tokenized_vault(&self) -> bool {  
        matches!(self, AccountType::TokenizedVault | AccountType::SingleAssetVault)  
    }  
}
```

RUST

And replace the old `if type == TokenizedVault` with `if type.is_tokenized_vault()`.

Another helper `supports_mint_init()` could replace `account_type != AccountType::Vault` with `account_type.supports_mint_init()` in `validate_args` and any other relevant call sites.

Developer Response:

The recommended sub vault type isn't backward compatible. We implemented the alternative vault type check helper function. Added `supports_mint()` helper function for vault type check.

E02: Invalid ATA owner priced as zero

Location: https://github.com/glamsystems/glam/blob/f888eeee5273476146438e9782e8356cf3db22435/anchor/programs/glam_mint/src/instructions/price_single_asset_vault.rs#L42-L59

```
> _price_single_asset_vault.rs RUST
40:     0, // Use 0 as placeholder bitflag (vault tokens don't have a specific protoco
41:     1)
42: );
43: if base_asset_ata.owner.eq(base_asset_token_program) {
44:     let data = base_asset_ata.try_borrow_data()?;
45:     let ata = TokenAccount::try_deserialize(&mut &data[..])?;
46:     // Validate mint and owner, explicit validation would be more defensive
47:     require_keys_eq!(
48:         ata.mint,
49:         glam_state.base_asset_mint,
50:         GlamError::InvalidTokenAccount
51:     );
52:     require_keys_eq!(
53:         ata.owner,
54:         ctx.accounts.glam_vault.key(),
55:         GlamError::InvalidTokenAccount
56:     );
57:     priced_protocol.amount = ata.amount.cast()?;
58: }
59:
60:
61: glam_emit_event!(PricedProtocolRecord {
```

Description:

If `base_asset_ata` is not owned by the token program (eg, closed or wrong account), pricing skips parsing and leaves the amount at 0.

Potential Benefit:

Clearer invariant and easier auditing; avoids silent zero pricing on invalid accounts.

Recommendation:

Require `base_asset_ata.owner == base_asset_token_program` or error on invalid ownership. If a closed ATA should be treated as zero, make that path explicit.

Developer Response:

Expected behavior. It handles first time pricing when the ATA address is valid but the token account doesn't exist yet. Added a comment to the code to document the decision explicitly.

E03: SingleAssetVault decimals are not constrained on init

Location: https://github.com/glamsystems/glam/blob/f888eee5273476146438e9782e8356cf3db22435/anchor/programs/glam_mint/src/instructions/initialize_mint.rs#L320-L328

```
> _initialize_mint.rs RUST
318:     GlamError::InvalidIxArgs
319: );
320: if let Some(share_decimals) = share_decimals {
321:     require!(share_decimals <= 9, GlamError::InvalidIxArgs);
322:     if account_type == AccountType::TokenizedVault {
323:         require!(
324:             share_decimals >= base_asset_decimals,
325:             GlamError::InvalidIxArgs
326:         );
327:     }
328: }
329: Ok(())
330: }
```

Description:

Decimals are unchecked for **SingleAssetVault**, so it can be initialized with incompatible **share_decimals** vs **base_asset_decimals**, making the vault unusable.

Potential Benefit:

Ensure **shares_decimals** >= **base_asset_decimals**.

Recommendation:

Check the decimals for **SingleAssetVault** in the same way as for **TokenizedVault**.

Developer Response:

Recommendation implemented.

E04: `system_transfer` is not blocked for `SingleAssetVault`

Location: https://github.com/glamsystems/glam/blob/f888eee5273476146438e9782e8356cf3db22435/anchor/programs/glam_protocol/src/instructions/system.rs#L51-L56

```
>_ system.rs RUST
49:         )?;
50:     } else {
51:         // To proceed, the vault must not be tokenized and the destination must
           be allowlisted
52:         require!(
53:             glam_state.account_type != AccountType::TokenizedVault
54:             && allowlist.contains(&token_account.owner),
55:             GlamError::UnauthorizedSigner
56:         );
57:         check_access_any(
58:             glam_state,
```

Description:

`SingleAssetVault` is tokenized but is not included in the `TokenizedVault` restriction here. If intentional, the comment should say so; if not, extend the restriction.

Potential Benefit:

Improve readability and reduce ambiguity around intended policy.

Recommendation:

Either update the comment to emphasize it only blocks `TokenizedVault`, or include `SingleAssetVault` if the intended policy is to restrict all tokenized vaults.

Developer Response:

Recommendation implemented.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)

- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.